无线车联网仿真

本章主要介绍无线车联网的主要技术、组成及应用场景，并利用 SUMO、OMNeT++等工具构建无线车联网仿真环境，进行简单的分析。旨在让读者能通过本章内容学习快速进入无线车联网仿真领域。

8.1 预备知识

8.1.1 车联网概述

车联网（Internet of Vehicles, IoV），也称车辆互联网或车辆与一切（Vehicle to Everything, V2X）通信，以车内网、车际网和车载移动互联网为基础，实现车辆互联以及与周围基础设施、人员和网络互联的高级系统。IoV 是应用于交通领域的更广泛的物联网生态系统的一个关键组成部分，其能够提高道路安全性、交通效率和整体旅行体验。图 8.1 所示为车联网部署示意图。

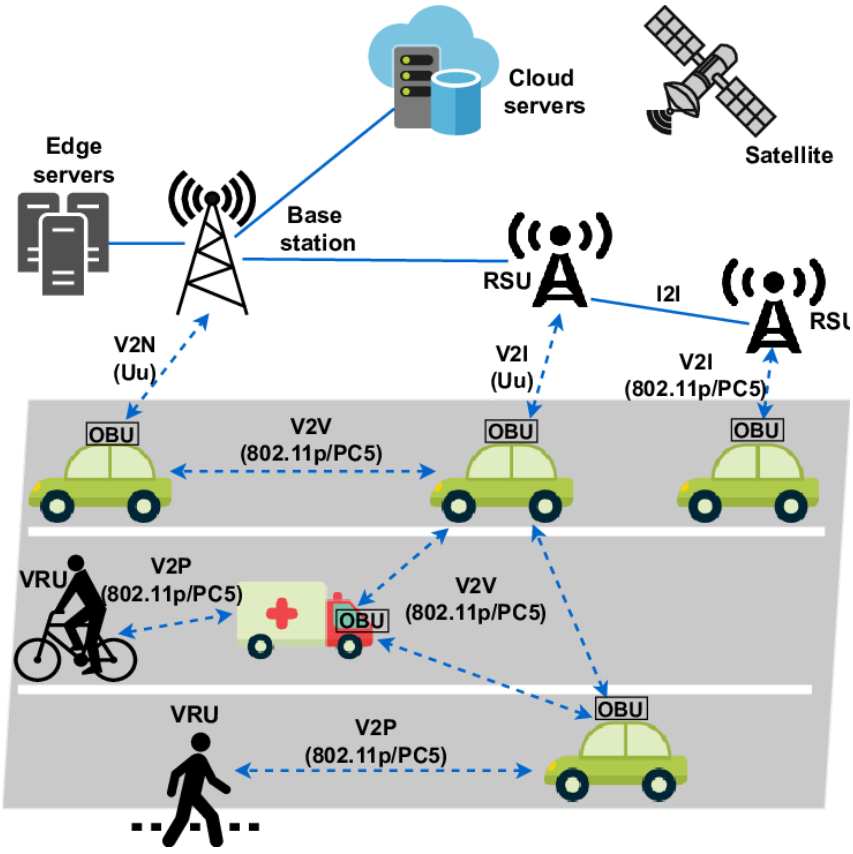


图 8.1　无线车联网部署示意图

由上图可知，车联网有两个关键单元组成，即车载单元（On-Board Unit, OBU）和路侧单元（Road Side Unit, RSU）。OBU 主要采集车况、路况、行人信息，提供与 RSU 及其他 OBU

的通讯信息交互功能，同时具有移动网络接入能力，接入车联网管理平台或云平台。RSU 则主要采集当前道路状况、交通状况等信息，通过通讯网络，与路侧感知设备、交通信号灯、电子标牌等终端通信，实现车路互联互通、交通信号实时交互等功能，辅助驾驶员进行驾驶，保障整个交通领域的人员及车辆安全。

在车联网实体间通信方面，主要有五种，即车与车（Vehicle-to-Vehicle, V2V）、车与人（Vehicle-to-Pedestrian, V2P）、车与路边基础设施（Vehicle-to-Infrastructure, V2I）、车与网络（Vehicle-to-Network, V2N）和车与云平台（Vehicle-to-Cloud, V2C）。V2V 交换位置、速度等信息，提高行车安全；V2P 旨在减少交通事故，保护行人安全；V2I 与交通信号灯、道路传感器等通信，管理和优化交通；V2N 用于获取实时交通信息、天气预报等；V2C 用于存储、处理和分析大量数据。

8.1.2 车联网通信技术

在车联网领域，主流的通信技术有两个，即专用短程通信（Dedicated Short-Range Communication, DSRC）和基于蜂窝的 V2X（Cellular V2X, C-V2X）。DSRC 和 C-V2X 在工作原理上存在较大的差异，表 8.1 是 DSRC 和 5G 蜂窝的对比。

表 8.1　DSRC 与 5G 蜂窝对比

对比指标	DSRC	5G 车联网
通信方式	IEEE 802.11p/IEEE 1609 标准通信	D2D 终端直连
最大传输距离	800m	1000m
最大移动速度	60km/h	350km/h
最大数据速率	27Mbps	1Gbps
频段	5.86~5.92GHz	授权频段
时延	大于 10ms	约 1ms

DSRC 由 IEEE 根据 Wi-Fi 标准制定，标准化流程始于 2004 年。DSRC 系统包含 OBU 与 RSU 两个重要组件，通过 OBU 与 RSU 提供车间与车路间信息的双向传输，RSU 再透过光纤等网络将交通信息传送至后端智能运输系统平台。

C-V2X 包括 OBU、RSU、Uu 接口和 PC5 接口。RSU 在覆盖范围内广播路况/信号灯/行人等信息，进行时间及位置同步等，接入车联网管理平台或云平台；Uu 接口是 OBU/RSU 与基站之间的接口，实现与移动网络通信；PC5 接口是 OBU 与 OBU，OBU 与 RSU 之间的直连通信接口，即车辆与其他设施间不借助移动网络而进行通信。

基于蜂窝通信的 C-V2X 相比 DSRC 在容量、时延、可管理性和抗干扰等多个方面更具优势，能覆盖约两倍的范围，或在相同范围内实现更高可靠性。在高速公路场景下（140-250km/h），

C-V2X 通信距离比 DSRC 提升了约 100%。在城市道路场景下（15-60km/h），C-V2X 通信距离比 DSRC 提升了约 30%。此外，C-V2X 还支持集中和分布式相结合的拥塞控制机制，显著提升高密场景下接入系统的用户数，实现更加高效的资源分配。从部署上看，C-V2X 可复用 4G/5G 基站和通信网络，部署成本更低。基于 802.11p 的 DSRC 组网需新建大量路侧单元，其硬件产品成本也比较高昂。C-V2X 前向兼容 5G，是唯一一种具有清晰 5G 演化路径的 V2X 技术。C-V2X 包含 LTE-V2X、eLTE-V2X 和向后演进的 5G NR-V2X。

8.2 实验环境

在本章实验中，将采用开源的 Veins 框架[1]（如图 8.2 所示，各主要模块细节请参考项目模块文档[2]），模拟 V2X 通信，特别是 IEEE 802.11p 和 ITS-G5 标准的车载通信网络。Veins 基于两个成熟的模拟器：①基于事件的网络模拟器 OMNeT++；②道路交通模拟器 SUMO。Veins 对这些模拟器进行了扩展，为 IVC 模拟提供了一套全面的模型。

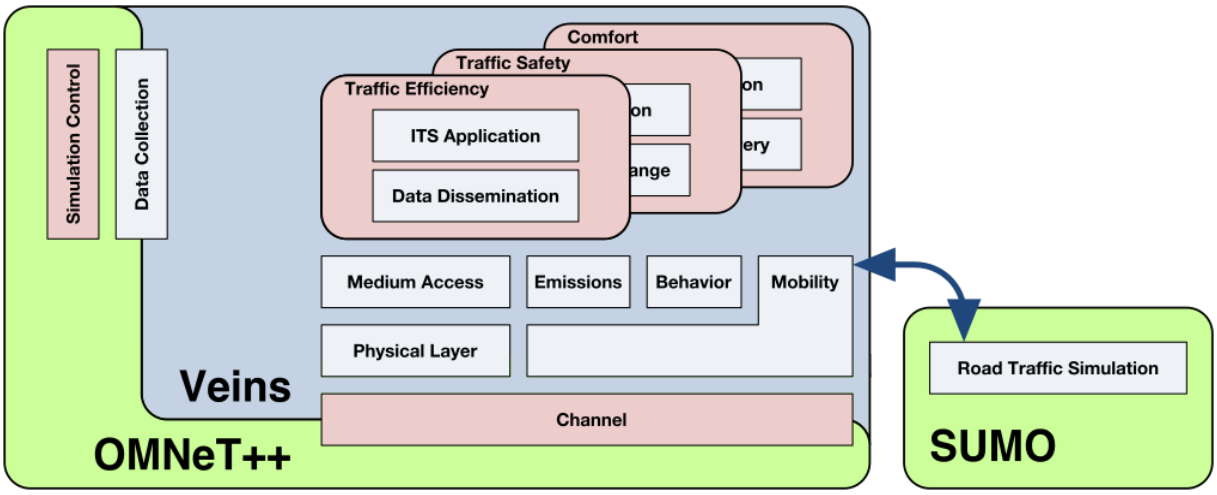


图 8.2　Veins 仿真框架示意图

Veins 主要特点包括：①集成了网络层和物理层仿真，允许研究者在一个统一的框架内模拟车辆移动和网络通信；②模块化设计，允许用户轻松扩展和修改仿真模型，以适应不同的研究需求；③开源和免费，利于学术研究和工业原型设计；④社区支持，提供文档和支持。

实验涉及的主要组件包括：①OMNeT++，提供网络层仿真，模拟数据包在网络中的传输和处理；②SUMO，提供车辆移动模型，模拟车辆在道路网络中的行驶；③交通流控制接口（Traffic Control Interface, TraCI）是 SUMO 的一个组件，允许 OMNeT++与 SUMO 进行实时通信，交换车辆位置和速度等信息。

在本章实验开始前，假定读者已在 VirtualBox 6.1.50 中安装了 Ubuntu 20.04.1 的虚拟机系

[1] https://veins.car2x.org/
[2] https://veins.car2x.org/documentation/modules/

统。并且用户名和主机名都为 **wiotlab**。

8.3 实验步骤

8.3.1 Veins 环境构建

(1)OMNeT++安装

在该实验中，OMNeT++安装包为“omnetpp-5.6.2-src-linux.tgz”（版本 5.6.2）[3]，其他版本未测试。具体安装过程如下：

①逐行执行下述命令，安装 OMNeT++编译所需的依赖包。

```
#1)更新软件源
sudo apt-get update
#2)安装编译依赖包
sudo apt-get install build-essential cmake gcc g++ bison flex perl python3 qt5-default libqt5opengl5-dev tcl-dev tk-dev libxml2-dev zlib1g-dev default-jre doxygen graphviz libwebkitgtk-3.0-0 libpcap-dev
#3)安装 osgearth 开发包
sudo apt-get install openscenegraph-plugin-osgearth libosgearth-dev
#4)并行模拟支持包
sudo apt-get install openmpi-bin libopenmpi-dev
```

②将安装包拷贝至虚拟机系统“**/home/wiotlab/WIoTLab/tools/veins**”目录中解压，然后将文件夹名字改为“**omnetpp**”。

③在终端中执行“**sudo gedit ~/.bashrc**”打开环境变量配置文件，并在文件最后添加“**export PATH=$PATH: /home/wiotlab/WIoTLab/tools/veins/omnetpp/bin**”。

④在终端中使用 **cd** 命令进入“**/home/wiotlab/WIoTLab/tools/veins/omnetpp**”目录，并执行“**./configure**”进行编译配置。紧接着，执行“make”编译 OMNeT++。成功编译后会提示“Now you can type "omnetpp" to start the IDE”。若希望从应用程序启动器或通过桌面快捷方式访问 IDE，在终端执行“**make install-menu-item; make install-desktop-icon**”。

⑤打开 OMNeT++，会弹出 INET 框架安装的提示框。勾选提示框中的两个复选框，并点击“OK”开始安装（该过程需要一定时间）。

(2)SUMO 安装

在该实验中，SUMO 的版本为 1.8.0[4]，直接从官网下载（sumo-src-1.8.0.zip），其他版本未测试。具体安装过程如下：

①在终端中执行“**sudo apt-get install libgdal-dev libproj-dev libxerces-c-dev**

[3] https://omnetpp.org/download/old
[4] https://sourceforge.net/projects/sumo/files/sumo/

libfox-1.6-dev”安装 SUMO 必要依赖库。

②将安装包拷贝至虚拟机系统“**/home/wiotlab/WIoTLab/tools/veins**”目录中解压，然后将文件夹名字改为“**sumo**”。

③在终端中执行“**sudo gedit ~/.bashrc**”打开环境变量配置文件，并在文件最后添加以下内容：

```
export SUMO_HOME=/home/wiotlab/WIoTLab/tools/veins/sumo
export PATH=$PATH: /home/wiotlab/WIoTLab/tools/veins/sumo/bin:
/home/wiotlab/WIoTLab/tools/veins/sumo/tools
```

④逐行执行下述命令，编译 SUMO 工具。编译成功后，可运行“**sumo-gui**”测试是否能够正常运行。若正常，会出现图 8.3 所示的界面。

```
cd /home/wiotlab/WIoTLab/tools/veins/sumo/build/cmake_modules
cmake ../..
make
```

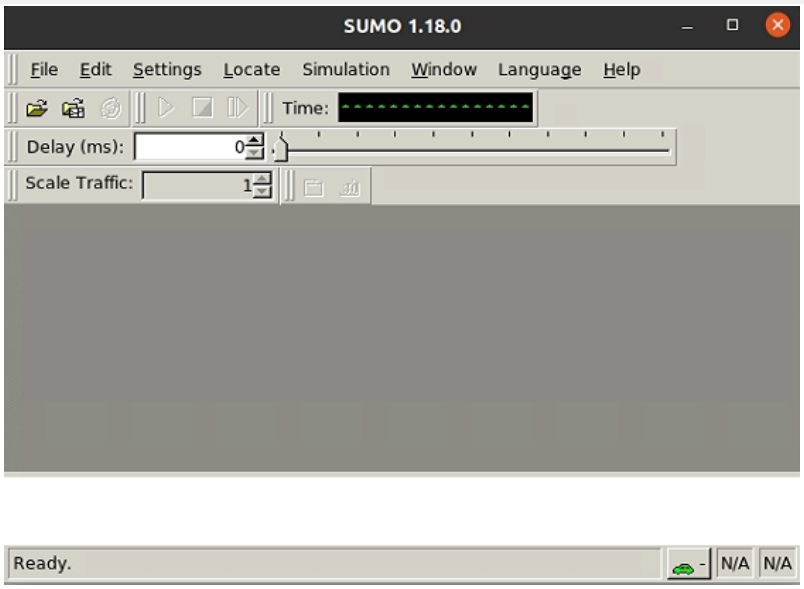


图 8.3　sumo-gui 界面

(3)Veins 安装

在该实验中，Veins 的版本为 5.2[5]，直接从官网下载（veins-5.2.zip），其他版本未测试。具体安装过程如下：

①在“**/home/wiotlab/WIoTLab/tools/veins/omnetpp/samples**”目录中建立一个文件夹“vanet”，并在该文件夹中建立“lte”和“ve”两个子文件夹。将 veins-5.2.zip 拷贝至“ve”中并解压，然后将其重命名为 **veins**。

②打开命令行终端，并使用 cd 命令切换至“**/home/wiotlab/WIoTLab/tools/veins/**

[5] https://veins.car2x.org/download/

omnetpp/samples/vanet/ve/veins”目录，然后执行“**./configure; make -j$(nproc); python2 sumo-launchd.py -vv -c sumo-gui**”，可得到最后输出“**Listening on port 9999**”（端口号 9999 与后面 **omnetpp.ini** 配置文件里一致），保持该窗口不要关闭。打开 OMNeT++，并导入目录“**/home/wiotlab/WioTLab /tools/veins/omnetpp/samples/vanet/ve/veins**”中的项目，然后按图 8.4 所示运行 Veins 例子（执行 **omnetpp.ini** 文件，即双击图中①的文件，会在右侧显示文件内容，其为仿真项目的配置信息。然后，点击②所示的绿色按钮即可启动仿真。）。

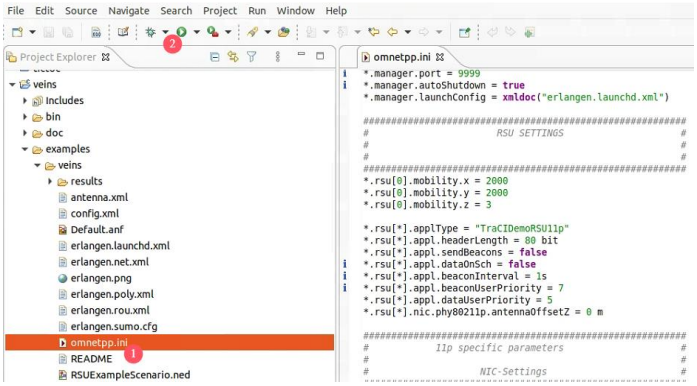


图 8.4　运行 Veins 例子

执行成功后，会出现图 8.5 所示界面，①处可选择实验配置文件（本章实验保持 Default 不变），然后点击②的 OK 按钮即可打开仿真界面（如图 8.6 所示）。

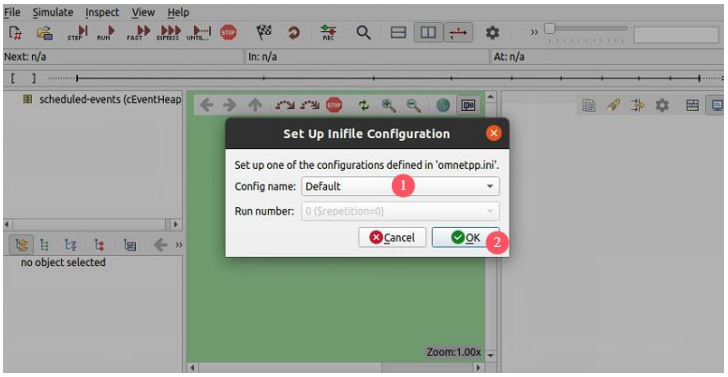


图 8.5　选择实验配置文件

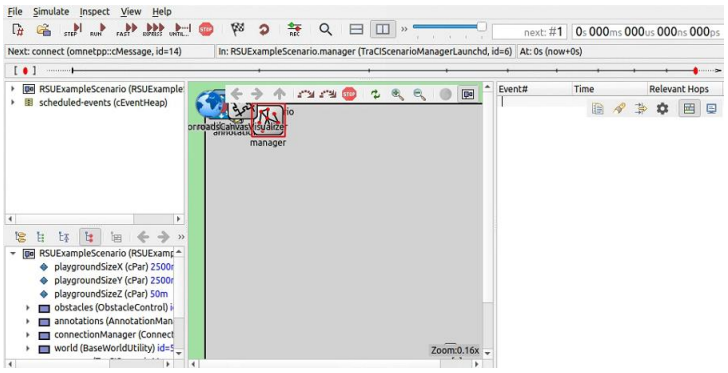


图 8.6　打开仿真界面

点击图 8.7 的①所示 RUN 按钮，即可开始仿真。

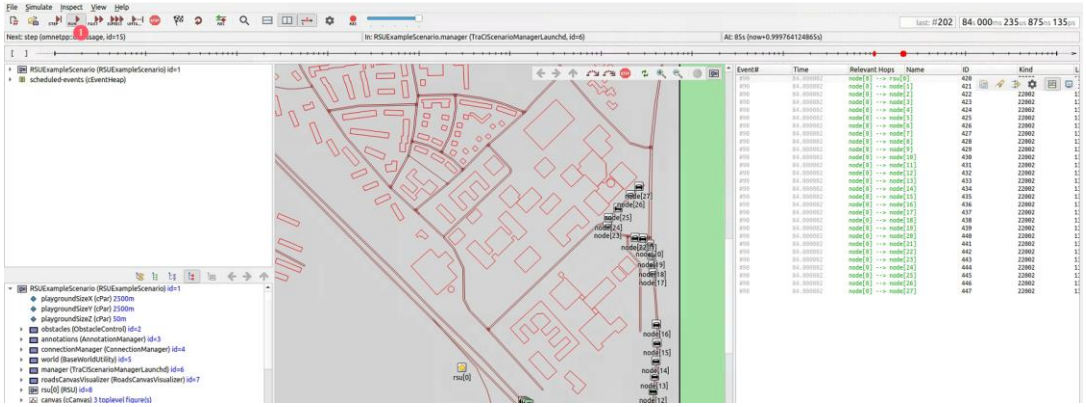


图 8.7　开始执行仿真

8.3.2 制作仿真地图

SUMO 仿真器运行需要有 3 个文件（以 lab8 命名）：**lab8.net.xml** 文件，构建路网、创建交通信号灯等；**lab8.rou.xml** 文件，生成车辆；**lab8.sumo.cfg** 文件，将 **lab8.net.xml** 和 **lab8.rou.xml** 文件结合起来实现仿真。SUMO 路网文件包含道路、交叉口 ID 和位置信息、车道信息（数量、长度、最大速度、形状、功能等）、优先权信息、交通信号信息、交叉口信息等。接下来将介绍如何定制路网文件，并应用到实际的仿真中。

从 OpenStreetMap 中导出 **lab8.osm** 地图文件，并利用 **sumo/bin** 目录下的 **netconvert**（生成道路文件）和 **polyconvert**（生成地形文件）转换工具，以及 **sumo/tools** 目录下的 **randomTrips.py**（生成车辆行为文件）工具生成路网文件，具体参考第 3 章 3.1.5 节内容（lab3 改为 lab8 即可），此处不再详述。正确执行后，会生成 **lab8.net.xml**、**lab8.poly.xml** 和 **lab8.rou.xml** 文件。与此同时，编写如下内容的 **lab8.sumo.cfg** 文件。

```
<?xml version="1.0" encoding="iso-8859-1"?>
<configuration xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="http://sumo.sf.net/xsd/sumoConfiguration.xsd">
<input>
    <net-file value="lab8.net.xml"/>
    <route-files value="lab8.rou.xml"/>
    <additional-files value="lab8.poly.xml"/>
</input>
<time>
    <begin value="0"/>
    <end value="1000"/>
    <step-length value="0.1"/>
</time>
<report>
```

```
    <no-step-log value="true"/>
</report>
<gui_only>
    <start value="true"/>
</gui_only>
</configuration>
```

执行命令 **sumo-gui lab8.sumo.cfg**，正确情况下会出现图 8.8 所示界面，可验证上述生成的路网文件是否正确。

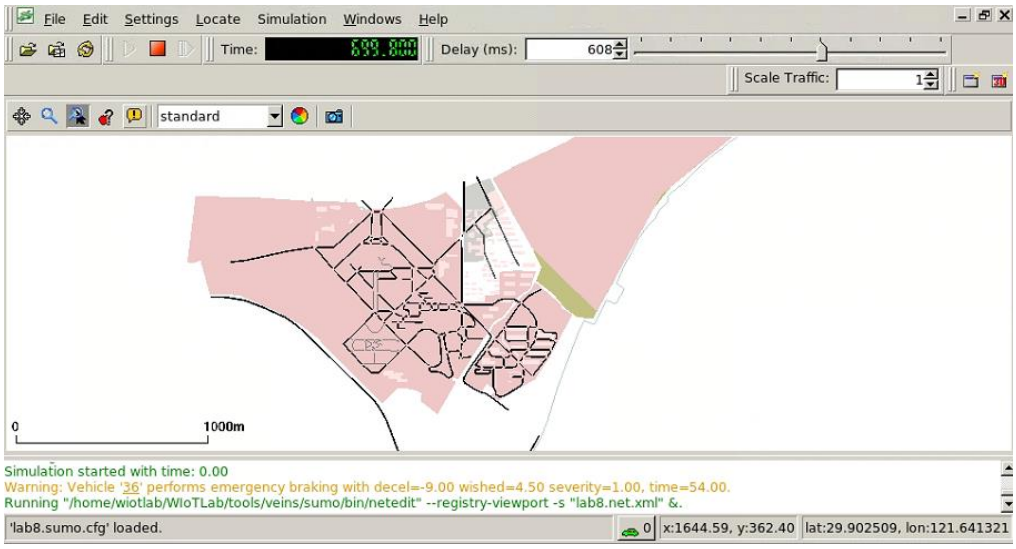


图 8.8　使用配置文件启动 SUMO 仿真

若需要对地图进行少量修改，可点击图 8.8 中的 **Edit** 后选择 **Open in netedit**，出现图 8.9 所示的界面。选择图中右边地图的路段（图中①），在左侧会出现②所示的可编辑信息。**注意：**修改后，**lab8.net.xml** 文件会发生变化，需重新生成车流行为文件，避免报错。

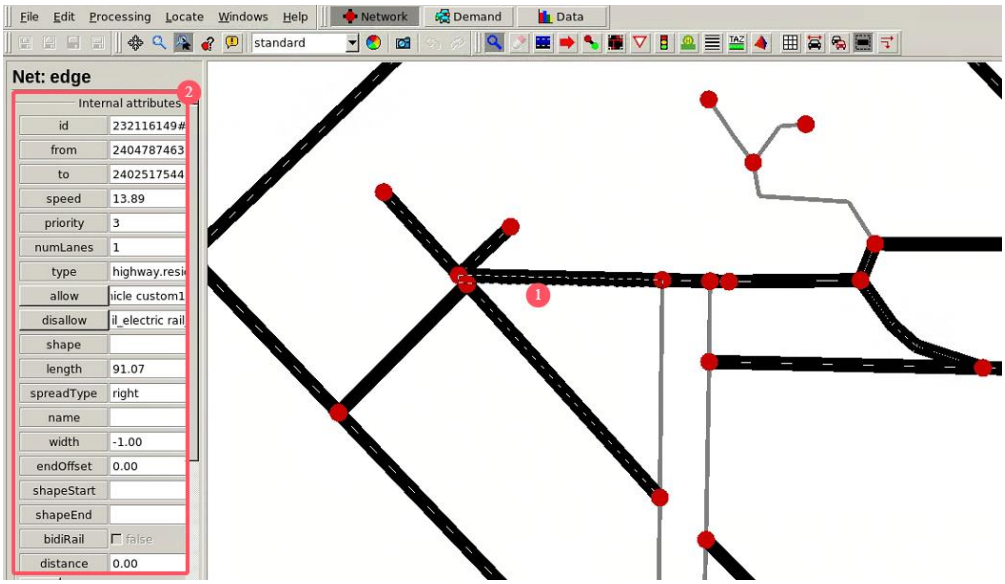


图 8.8　netedit 地图编辑界面

8.3.3 Veins 加载新地图

在将地图加入 Veins 前，需编写 lab8.launchd.xml 文件，其内容如下所示：

```
<launch>
    <copy file="lab8.net.xml" />
    <copy file="lab8.rou.xml" />
    <copy file="lab8.poly.xml" />
    <copy file="lab8.sumo.cfg" type="config" />
</launch>
```

将 lab8.launchd.xml、lab8.net.xml、lab8.rou.xml、lab8.poly.xml 和 lab8.sumo.cfg 五个文件拷贝至“**/home/wiotlab/WIoTLab/tools/veins/omnetpp/samples/vanet/ve/veins/examples/veins**”目录，并修改其中的 omnetpp.ini 文件（将 xmldoc("erlangen.launchd.xml") 修改为 xmldoc("lab8.launchd.xml")）。紧接着，按图 8.4 所示运行 Veins。

在设计仿真场景时，可能会涉及到多个 RSU，需在目录 **veins/examples/veins** 中的 RSUExampleScenario.ned 文件中设置 RSU 数量，且在 **veins/examples/veins** 目录中的 omnetpp.ini 文件中设置每个 RSU 相对于地图中的位置。

8.3.4 Veins 代码解析

(1)目录 veins/examples/veins 中文件

- results 文件夹：存储模拟运行后生成的所有输出和日志文件。
  - Scalar files(.sca)：包含模拟结束时的统计信息，如发送和接收的包数量，延迟平均值和标准差等。
  - Vector files(.vec)：包含模拟过程中记录的时间序列数据，如每个包发送和接收时间，每个节点位置等。
  - Eventlog files(.elog)：若启用了事件日志记录，这些文件包含模拟过程中所有事件详细信息，可用 OMNeT++的 Sequence Chart 工具查看。
  - Log files(.log)：包含模拟过程中的日志信息，如错误和警告信息。
- antenna.xml：用于定义和配置无线通信中的天线模型。
- config.xml：用于配置模拟的无线信号传播和接收模型的物理特性。AnalogueModels 部分包含两个 AnalogueModel：SimplePathlossModel 和 SimpleObstacleShadowing。
  - SimplePathlossModel 有一个 alpha 参数，值为 2.0。alpha 通常用于描述路径损失模型中的路径损失指数，它是一个描述随着距离增加，信号强度如何衰减的参数。
  - SimpleObstacleShadowing 是一个描述物体（如建筑物）对无线信号造成阴影效应

的模型。这里定义了一种类型的障碍物 building，并给出了每次穿越和每米对信号强度造成的衰减。

- Decider 部分定义了一个类型为 Decider80211p 的决策器。决策器在无线通信中通常用于决定在给定的信号和噪声条件下，是否可以成功接收一个包。在这里，Decider80211p 有一个 centerFrequency 参数，值为 5.890e9，这个参数表示决策器监听的中心频率。

- erlangen.launchd.xml：用于配置 SUMO 的启动参数。通过修改这个文件，你可以改变模拟的输入、时间参数、报告设置和 GUI 行为，从而影响模拟的行为和结果。
- erlangen.net.xml：定义了网络文件，描述了道路网络的结构。
- erlangen.poly.xml：定义了附加文件，可能用于描述建筑物或其他地理特征。
- erlangen.rou.xml：定义了路线文件，描述了车辆的路线。这里提供了车流控制的方法。
- erlangen.sumo.cfg：定义 SUMO 的输入文件、模拟的时间参数、报告设置以及 GUI 的行为。<input>部分定义了模拟的输入文件，<time>部分定义了模拟时间的参数(开始时间、结束时间、时间步长)，<report>部分定义了模拟报告的设置，<gui_only>部分定义了 GUI 的行为。
- omnetpp.ini：用于配置模拟参数的文件，可以定义和配置各种模拟参数，如模拟的持续时间、所使用的网络模型、模块参数、随机数生成器种子等。
- RSUExampleScenario.ned：描述了一个名为 RSUExampleScenario 的网络，该网络扩展自 Scenario 类，并包含一个 RSU 类型的子模块。更改仿真场景 RSU 数量时需要修改此文件。

(2)目录 veins/src/veins/modules/applicaton/traci 中部分文件

- TraCIDemo11p.ned：定义了一个名为 TraCIDemo11p 的简单模块，用于模拟车辆的应用层功能。该模块可以进行处理和发送基于车载网络的消息。
- TraCIDemo11p.cc：演示如何在车载网络中使用和处理消息。该模块实现了一个简单的车辆通信协议，其中车辆会定期广播其位置信息，并且可以处理接收到的其他车辆的位置信息。
- TraCIDemo11pMessage.msg：定义了一个 Veins 中的包，这个包可以在车载网络模拟中用于传输数据。
- TraCIDemoRSU11p.ned：定义了一个名为 TraCIDemoRSU11p 的简单模块，用于模拟

路侧单元（RSU）的功能。这个模块可以用于处理和发送基于车载网络的消息。

8.4 扩展实验*

本章介绍了无线车联网的基本概念、原理和通信方式，并利用 Veins 框架进行了分析，同时探讨了如何修改 Veins 代码实现自己的需求。在此基础上，读者可深入探索，扩展实验，如根据所在地区域，从 OpenStreetMap 截取合适的地图，并重复本章实验过程。有关 Veins 更多代码解释，可参考：https://github.com/francis1721/Veins-Tutorial。

参考文献

[1] 况博裕, 李雨泽, 顾芳铭, 等. 车联网安全研究综述：威胁、对策与未来展望. 计算机研究与发展, 2023, 60(10): 2304-2321.

[2] 陈山枝, 时岩, 胡金玲. 蜂窝车联网(CGV2X)综述. 中国科学基金, 2020, 34(2): 179-185.

[3] 金博, 胡延明. C-V2X 车联网产业发展综述与展望. 电信科学, 2020, 36 (3): 93-99.

[4] 林粤伟, 王溢, 张奇勋, 等. 面向 6G 的通信感知一体化车联网研究综述. 信号处理, 2023, 39(6): 963-974.

[5] 丁飞, 张楠, 李升波, 等. 智能网联车路云协同系统架构与关键技术研究综述. 自动化学报, 2022, 48(12): 2863−2885.

[6] Md. Noor-A-Rahim, Zilong Liu, Haeyoung Lee, et al. 6G for Vehicle-to-Everything (V2X) Communications: Enabling Technologies, Challenges, and Opportunities. Proceedings of the IEEE, 2022, 110(6): 712-734.

[7] Roshan Sedar, Charalampos Kalalas, Francisco Vázquez-Gallego, et al. A Comprehensive Survey of V2X Cybersecurity Mechanisms and Future Research Paths. IEEE Open Journal of the Communications Society, 2023, 4: 325-391.